

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
from google.colab import files

uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
df = pd.read_csv("Sample-Superstore.csv")

print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Dataset loaded successfully!
Rows: 9994
Columns: 22

```
df.head()
```

	csvbase_row_id	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	...	Postal Code	Region	Product ID	Cate
0	1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	42420	South	FUR-BO-10001798	Furr
1	2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	42420	South	FUR-CH-10000454	Furr
2	3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	...	90036	West	OFF-LA-10000240	C Sup
3	4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	33311	South	FUR-TA-10000577	Furr
4	5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	33311	South	OFF-ST-10000760	C Sup

5 rows × 22 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 22 columns):
#   Column              Non-Null Count  Dtype
---  -
0   csvbase_row_id      9994 non-null   int64
1   Row ID              9994 non-null   int64
2   Order ID            9994 non-null   object
3   Order Date          9994 non-null   object
4   Ship Date           9994 non-null   object
5   Ship Mode           9994 non-null   object
6   Customer ID         9994 non-null   object
7   Customer Name       9994 non-null   object
```

```
8 Segment 9994 non-null object
9 Country 9994 non-null object
10 City 9994 non-null object
11 State 9994 non-null object
12 Postal Code 9994 non-null int64
13 Region 9994 non-null object
14 Product ID 9994 non-null object
15 Category 9994 non-null object
16 Sub-Category 9994 non-null object
17 Product Name 9994 non-null object
18 Sales 9994 non-null float64
19 Quantity 9994 non-null int64
20 Discount 9994 non-null float64
21 Profit 9994 non-null float64
dtypes: float64(3), int64(4), object(15)
```

memory usage: 1.7+ MB

df.describe(include="all")

	csvbase_row_id	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	...	Postal Code	Regi
count	9994.000000	9994.000000	9994	9994	9994	9994	9994	9994	9994	9994	...	9994.000000	99
unique	NaN	NaN	5009	1237	1334	4	793	793	3	1	...	NaN	
top	NaN	NaN	CA-2017-100111	2016-09-05	2015-12-16	Standard Class	WB-21850	William Brown	Consumer	United States	...	NaN	W
freq	NaN	NaN	14	38	35	5968	37	37	5191	9994	...	NaN	32
mean	4997.500000	4997.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	55190.379428	N
std	2885.163629	2885.163629	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	32063.693350	N
min	1.000000	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1040.000000	N
25%	2499.250000	2499.250000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	23223.000000	N
50%	4997.500000	4997.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	56430.500000	N
75%	7495.750000	7495.750000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	90008.000000	N
max	9994.000000	9994.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	99301.000000	N

11 rows × 22 columns

df.isnull().sum()

```

      0
csvbase_row_id  0
      Row ID    0
      Order ID  0
      Order Date 0
      Ship Date  0
      Ship Mode  0
      Customer ID 0
Customer Name   0
      Segment   0
      Country   0
      City      0
      State     0
      Postal Code 0
      Region    0
      Product ID 0
      Category  0
      Sub-Category 0
      Product Name 0
      Sales     0
      Quantity  0
      Discount  0
      Profit    0

```

dtype: int64

```
print("Number of duplicate rows:", df.duplicated().sum())
```

Number of duplicate rows: 0

```
df = df.drop_duplicates()
```

```
print("Dataset shape after removing duplicates:", df.shape)
```

Dataset shape after removing duplicates: (9994, 22)

```
missing_values = df.isnull().sum()
```

```
print(missing_values)
```

```

csvbase_row_id    0
Row ID            0
Order ID          0
Order Date        0
Ship Date         0
Ship Mode         0
Customer ID       0
Customer Name     0
Segment          0
Country           0
City              0
State             0
Postal Code       0
Region           0
Product ID        0
Category          0
Sub-Category      0
Product Name      0
Sales             0
Quantity          0
Discount          0

```

```
Profit          0
dtype: int64
```

```
print(df.columns.tolist())
```

```
['csvbase_row_id', '\uffeffRow ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment']
```

```
df.describe()
```

	csvbase_row_id	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.656896
std	2885.163629	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	4997.500000	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

```
print("Total Sales:", df["Sales"].sum())
print("Total Profit:", df["Profit"].sum())
print("Average Sales:", df["Sales"].mean())
print("Average Profit:", df["Profit"].mean())
```

```
Total Sales: 2297200.8603000003
Total Profit: 286397.0217
Average Sales: 229.8580083049833
Average Profit: 28.65689630778467
```

```
print("Minimum Sales:", df["Sales"].min())
print("Maximum Sales:", df["Sales"].max())
```

```
Minimum Sales: 0.444
Maximum Sales: 22638.48
```

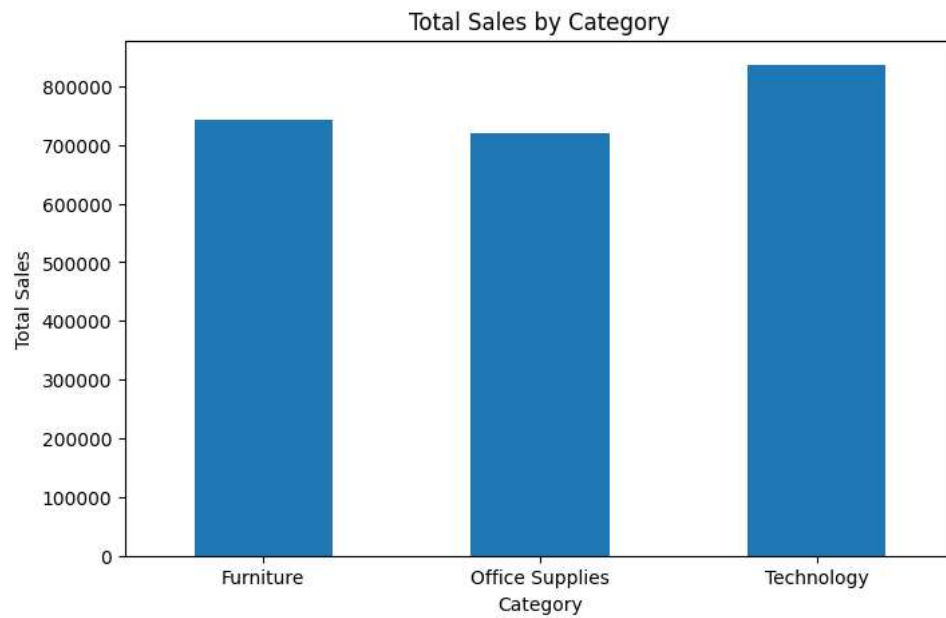
```
category_sales = df.groupby("Category")["Sales"].sum()
```

```
plt.figure(figsize=(8,5))
```

```
category_sales.plot(kind="bar")
```

```
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
```

```
plt.xticks(rotation=0)
plt.show()
```



```
category_profit = df.groupby("Category")["Profit"].sum()
```

```
plt.figure(figsize=(8,5))
```

```
category_profit.plot(kind="bar")
```

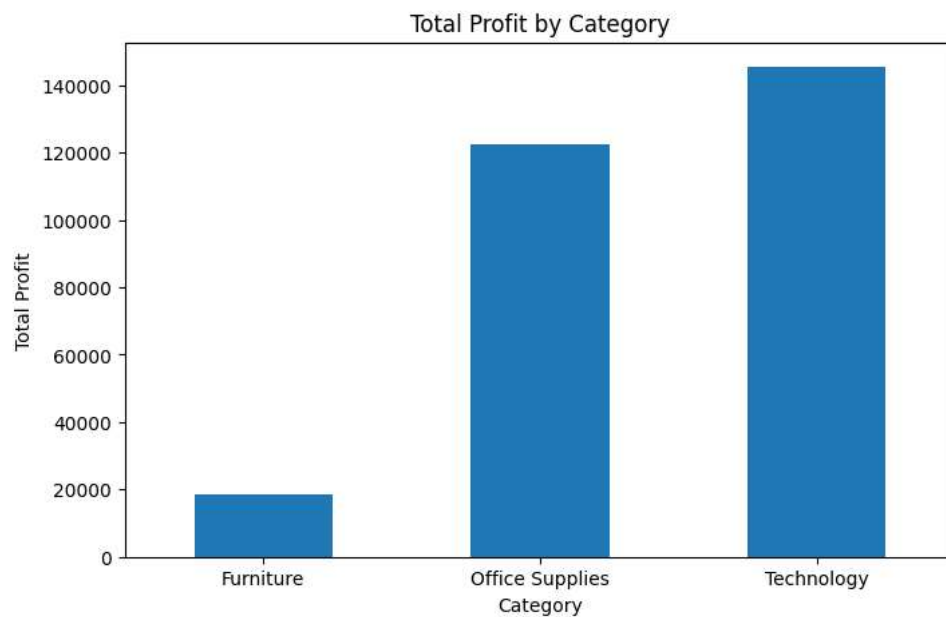
```
plt.title("Total Profit by Category")
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Total Profit")
```

```
plt.xticks(rotation=0)
```

```
plt.show()
```



```
region_sales = df.groupby("Region")["Sales"].sum()
```

```
plt.figure(figsize=(8,5))
```

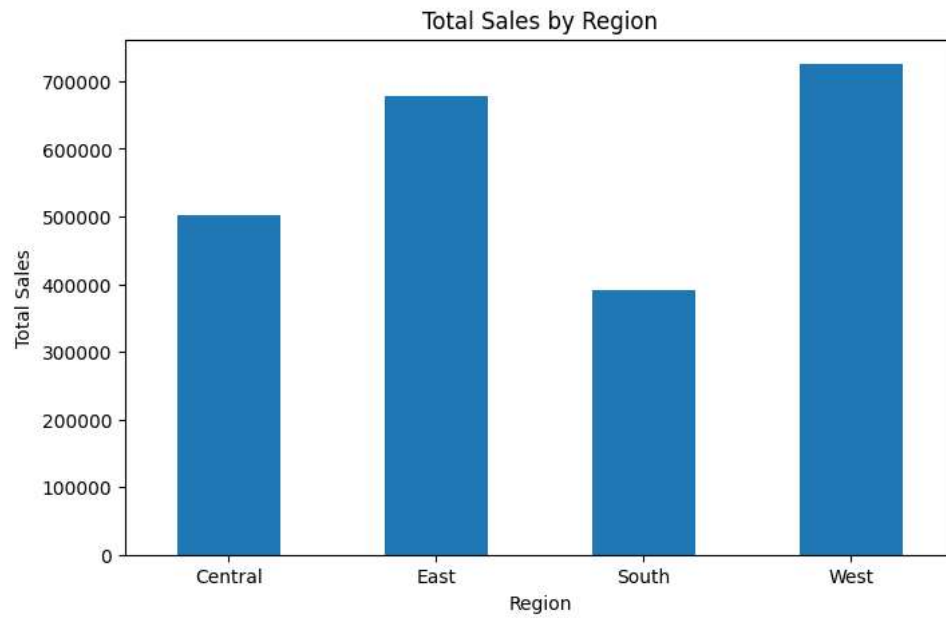
```
region_sales.plot(kind="bar")
```

```
plt.title("Total Sales by Region")
```

```
plt.xlabel("Region")
```

```
plt.ylabel("Total Sales")
```

```
plt.xticks(rotation=0)
plt.show()
```



```
df["Order Date"] = pd.to_datetime(df["Order Date"])
```

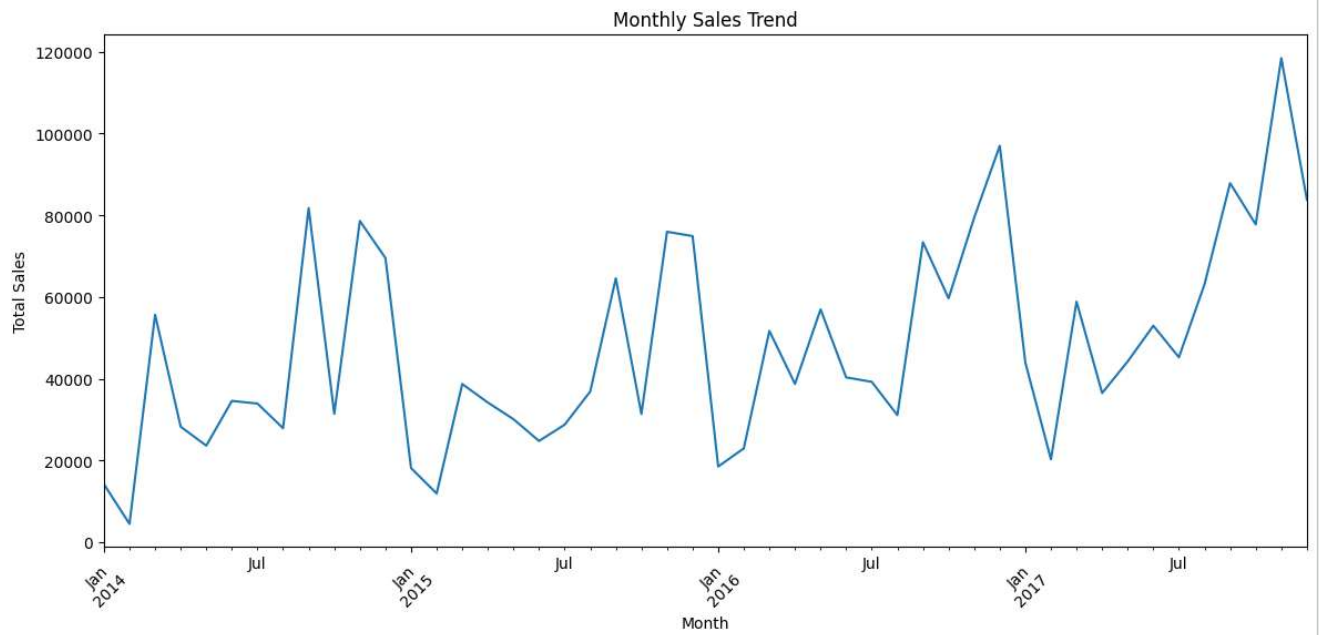
```
monthly_sales = df.groupby(
    df["Order Date"].dt.to_period("M")
)["Sales"].sum()
```

```
plt.figure(figsize=(14,6))
```

```
monthly_sales.plot()
```

```
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
```

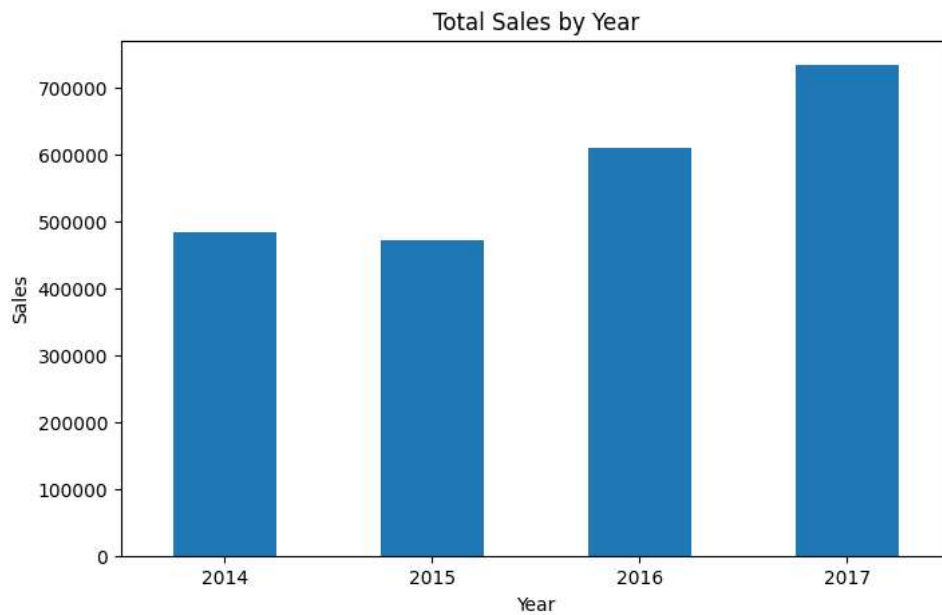
```
plt.xticks(rotation=45)
plt.show()
```



```
yearly_sales = df.groupby(  
    df["Order Date"].dt.year  
)["Sales"].sum()  
  
print(yearly_sales)
```

```
Order Date  
2014    484247.4981  
2015    470532.5090  
2016    609205.5980  
2017    733215.2552  
Name: Sales, dtype: float64
```

```
plt.figure(figsize=(8,5))  
  
yearly_sales.plot(kind="bar")  
  
plt.title("Total Sales by Year")  
plt.xlabel("Year")  
plt.ylabel("Sales")  
  
plt.xticks(rotation=0)  
plt.show()
```



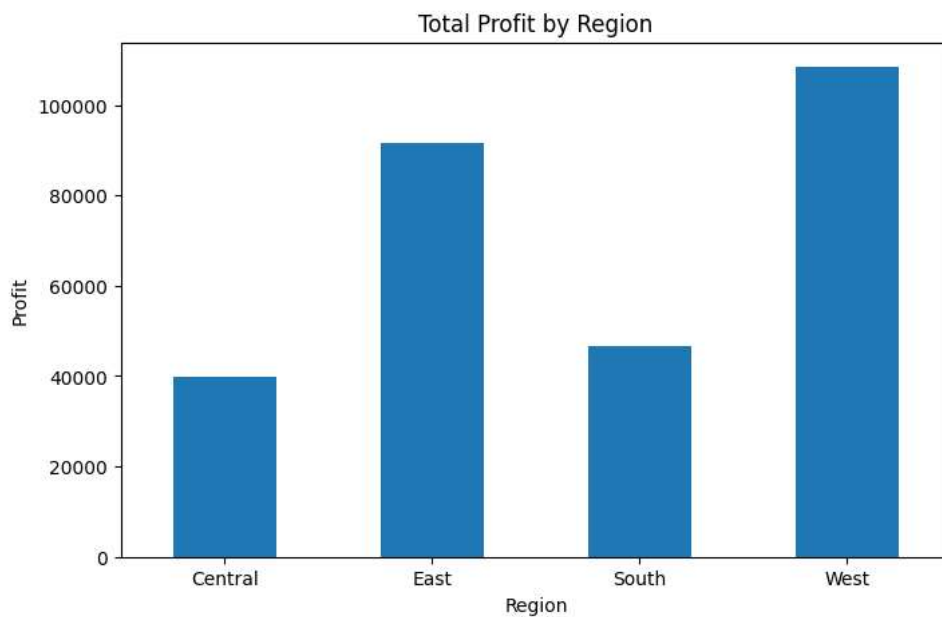
```
region_profit = df.groupby("Region")["Profit"].sum()

plt.figure(figsize=(8,5))

region_profit.plot(kind="bar")

plt.title("Total Profit by Region")
plt.xlabel("Region")
plt.ylabel("Profit")

plt.xticks(rotation=0)
plt.show()
```



```
category_summary = df.groupby("Category").agg({
    "Sales": "sum",
    "Profit": "sum"
}).sort_values("Profit", ascending=False)

category_summary
```

	Sales	Profit
Category		
Technology	836154.0330	145454.9481
Office Supplies	719047.0320	122490.8008
Furniture	741999.7953	18451.2728

```
top_products = df.groupby("Product Name")["Sales"].sum().sort_values(
    ascending=False
).head(10)
```

```
top_products
```

	Sales
Product Name	
Canon imageCLASS 2200 Advanced Copier	61599.824
Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	27453.384
Cisco TelePresence System EX90 Videoconferencing Unit	22638.480
HON 5400 Series Task Chairs for Big and Tall	21870.576
GBC DocuBind TL300 Electric Binding System	19823.479
GBC Ibimaster 500 Manual ProClick Binding System	19024.500
Hewlett Packard LaserJet 3310 Copier	18839.686
HP Designjet T520 Inkjet Large Format Printer - 24" Color	18374.895
GBC DocuBind P400 Electric Binding System	17965.068
High Speed Automatic Electric Letter Opener	17030.312

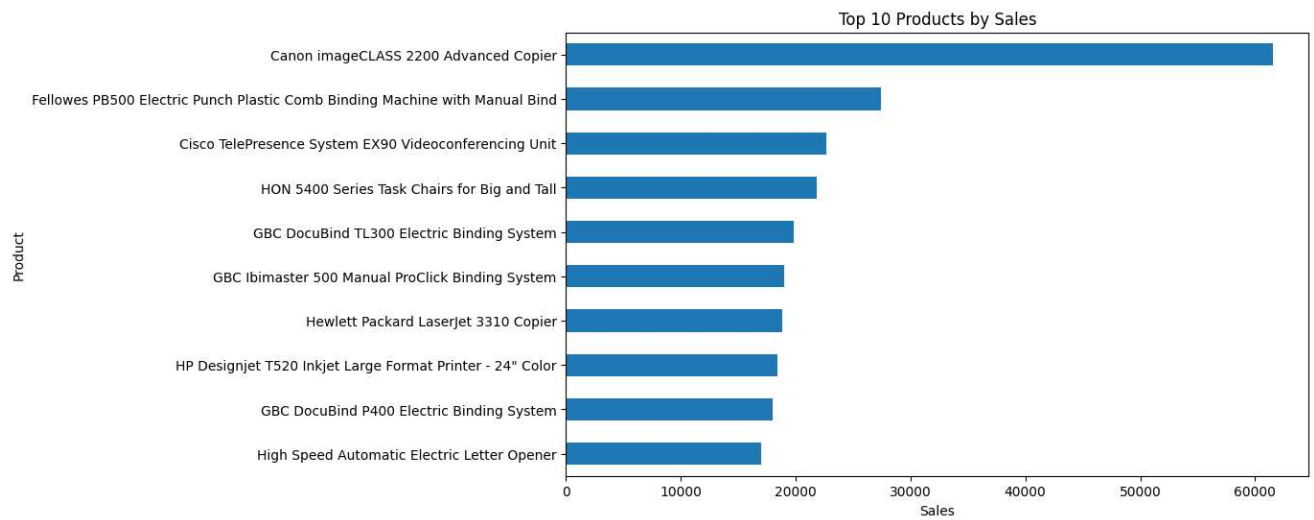
```
dtype: float64
```

```
plt.figure(figsize=(10,6))

top_products.sort_values().plot(kind="barh")

plt.title("Top 10 Products by Sales")
plt.xlabel("Sales")
plt.ylabel("Product")

plt.show()
```

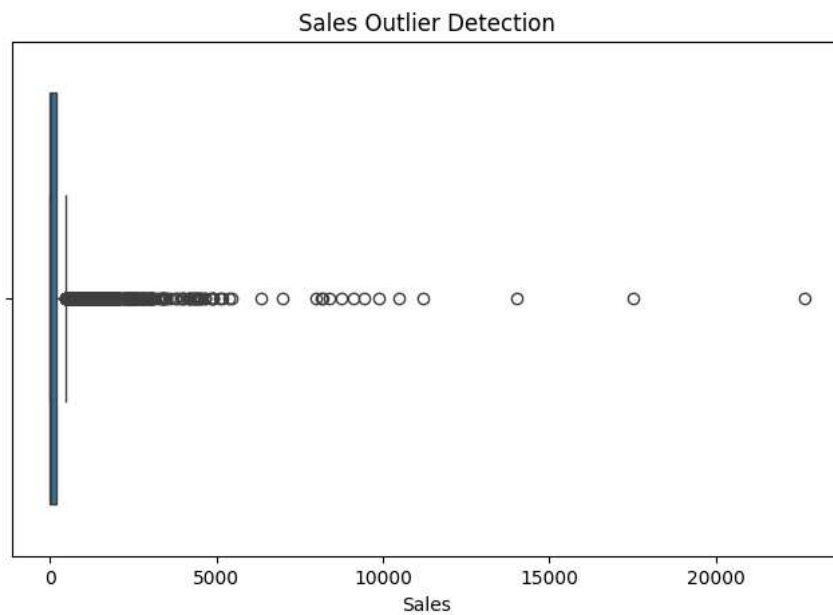


```
plt.figure(figsize=(8,5))

sns.boxplot(x=df["Sales"])

plt.title("Sales Outlier Detection")
plt.xlabel("Sales")

plt.show()
```



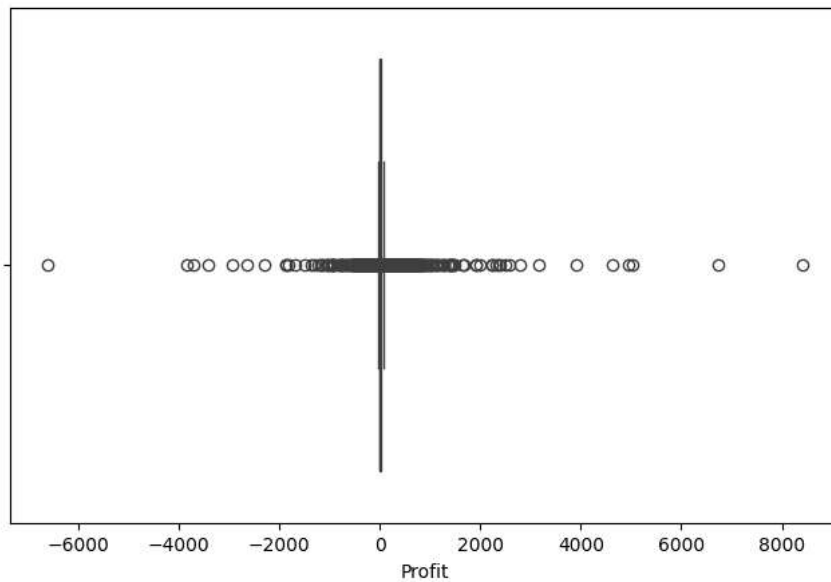
```
plt.figure(figsize=(8,5))

sns.boxplot(x=df["Profit"])

plt.title("Profit Outlier Detection")
plt.xlabel("Profit")

plt.show()
```

Profit Outlier Detection



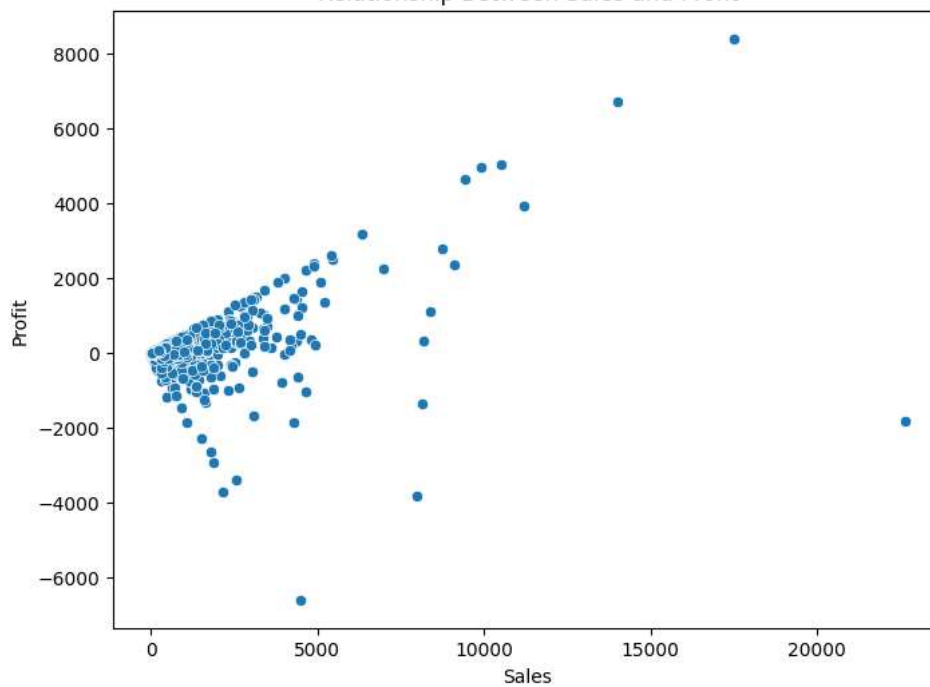
```
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit"
)

plt.title("Relationship Between Sales and Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")

plt.show()
```

Relationship Between Sales and Profit



```
numeric_df = df.select_dtypes(include="number")

correlation = numeric_df.corr()

correlation
```

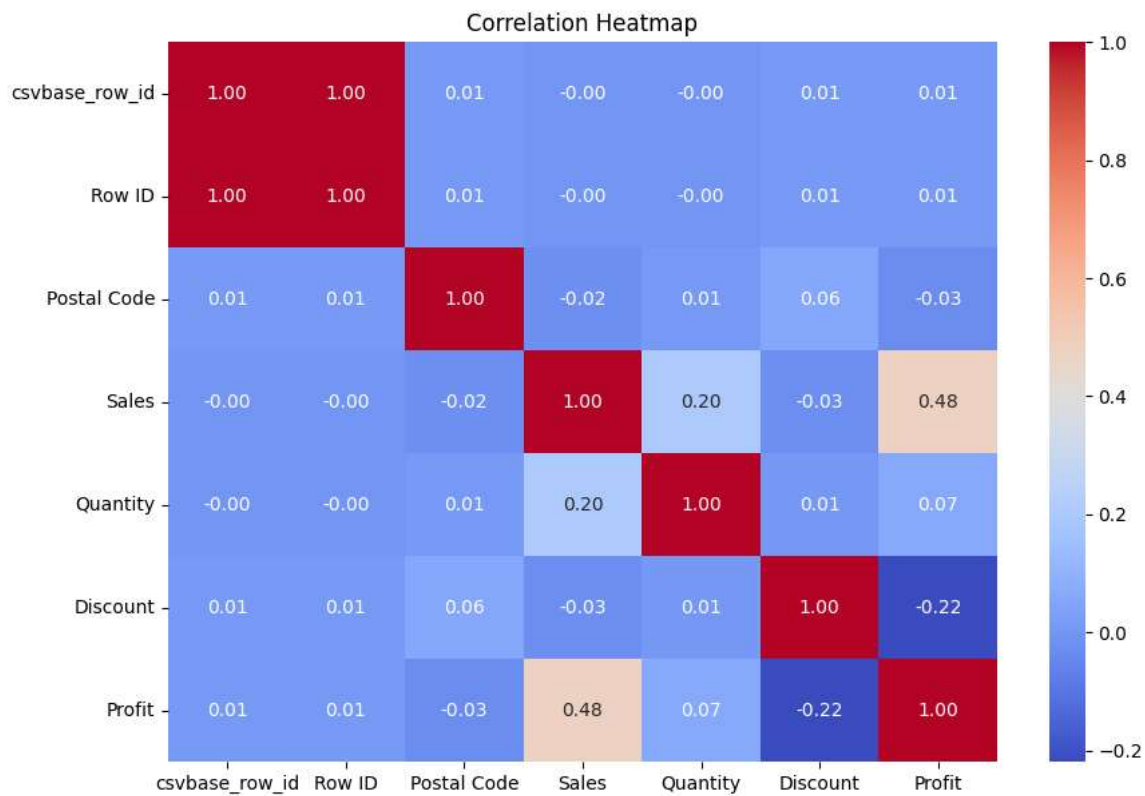
	csvbase_row_id	Row ID	Postal Code	Sales	Quantity	Discount	Profit
csvbase_row_id	1.000000	1.000000	0.009671	-0.001359	-0.004016	0.013480	0.012497
Row ID	1.000000	1.000000	0.009671	-0.001359	-0.004016	0.013480	0.012497
Postal Code	0.009671	0.009671	1.000000	-0.023854	0.012761	0.058443	-0.029961
Sales	-0.001359	-0.001359	-0.023854	1.000000	0.200795	-0.028190	0.479064
Quantity	-0.004016	-0.004016	0.012761	0.200795	1.000000	0.008623	0.066253
Discount	0.013480	0.013480	0.058443	-0.028190	0.008623	1.000000	-0.219487
Profit	0.012497	0.012497	-0.029961	0.479064	0.066253	-0.219487	1.000000

```
plt.figure(figsize=(10,7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```



```
highest_sales_category = df.groupby("Category")["Sales"].sum().idxmax()

print("Category with highest sales:", highest_sales_category)
```

Category with highest sales: Technology

```
highest_profit_category = df.groupby("Category")["Profit"].sum().idxmax()

print("Category with highest profit:", highest_profit_category)
```

Category with highest profit: Technology

```
highest_sales_region = df.groupby("Region")["Sales"].sum().idxmax()
```

```
Region with highest sales: West
```

```
highest_profit_region = df.groupby("Region")["Profit"].sum().idxmax()

print("Region with highest profit:", highest_profit_region)
```

```
Region with highest profit: West
```

```
final_summary = pd.DataFrame({
    "Metric": [
        "Total Sales",
        "Total Profit",
        "Average Sales",
        "Average Profit",
        "Highest Sales Category",
        "Highest Profit Category"
```